An toàn điện và đọc bản vẽ

Trước khi chạm vào tủ điện, bạn cần biết mình đang làm việc với mức điện áp nào. Một tủ điều khiển thường có ba cấp: **380/400 VAC 3 pha** cho động lực (động cơ, biến tần), **220/230 VAC** cho chiếu sáng và một số thiết bị, và **24 VDC** cho mạch điều khiển (PLC, cảm biến, nút nhấn, đèn báo).

Quy tắc bắt buộc

- **Cắt điện – khóa – treo biển (LOTO, Lock Out Tag Out)** trước khi sửa phần động lực. Ổ khóa là của bạn, chìa khóa nằm trong túi bạn.
- **Kiểm tra không còn điện** bằng đồng hồ đã thử trên nguồn đang có điện. Không tin vào việc "đã tắt aptomat".
- Biến tần có tụ DC còn điện vài phút sau khi cắt nguồn. Chờ đủ thời gian ghi trên nhãn thiết bị.
- Không đo điện trở hay thông mạch khi mạch đang có điện.

Đọc bản vẽ điện

Bản vẽ trong nhà máy thường theo chuẩn IEC (EPLAN là phần mềm phổ biến). Bạn sẽ gặp: **sơ đồ động lực** (aptomat → contactor → relay nhiệt → động cơ), **sơ đồ điều khiển** (nguồn 24 VDC, PLC, I/O), và **bảng đấu dây (terminal list)**. Mỗi thiết bị có mã tham chiếu như `-Q1` (aptomat), `-K1` (contactor/relay), `-S1` (nút nhấn), `-B1` (cảm biến), `-M1` (động cơ), `-H1` (đèn báo). Chú thích "/3.5" nghĩa là xem trang 3, cột 5.

> **Đi làm:** việc đầu tiên khi vào nhà máy là được giao bộ bản vẽ và đi dò tủ. Tập đọc bản vẽ sớm sẽ giúp bạn khác biệt so với các bạn thực tập khác.

Đo kiểm bằng đồng hồ vạn năng

Việc cần đo	Thang đo	Kết quả bình thường
Nguồn điều khiển	V DC	24 V giữa L+ và M (chấp nhận khoảng 20,4–28,8 V với S7-1200)
Nguồn 1 pha	V AC	Khoảng 220–230 V giữa L và N
Nguồn 3 pha	V AC	Khoảng 380–400 V giữa hai pha bất kỳ, ba cặp gần bằng nhau
Thông mạch dây, tiếp điểm	Ω hoặc còi	Gần 0 Ω. Chỉ đo khi đã cắt điện
Cuộn hút contactor	Ω	Vài chục đến vài trăm Ω. Vô cùng là đứt cuộn

Màu dây thường gặp theo IEC

Mạch động lực: pha L1 nâu, L2 đen, L3 xám, trung tính N xanh dương, dây bảo vệ PE vàng sọc xanh lá. Mạch điều khiển DC trong nhiều tủ: +24 V dùng dây xanh dương đậm, 0 V dùng xanh dương sọc trắng hoặc xanh dương nhạt. Mỗi nhà máy có thể quy ước riêng, nên luôn đối chiếu với trang "quy ước" ở đầu bộ bản vẽ thay vì đoán theo màu.

Ví dụ thực tế

Bạn được giao kiểm tra vì sao băng tải M3 không chạy. Trình tự đúng: mở bản vẽ, tìm trang có `-M3`, đi ngược lên thấy `-F3` (relay nhiệt), `-K3` (contactor), `-Q3` (aptomat). Ra tủ, nhìn nhãn dán trên thiết bị đúng với mã đó, thấy `-Q3` đang ở vị trí nhảy. Đó là điểm bắt đầu tìm nguyên nhân, không phải đóng lại ngay.

Lỗi người mới hay mắc

- Đóng lại aptomat vừa nhảy mà chưa tìm nguyên nhân (chạm đất, kẹt cơ khí, quá tải).
- Đo điện trở khi mạch còn điện, làm cháy cầu chì đồng hồ.
- Tin vào màu dây thay vì đo. Tủ cũ có thể đã bị sửa không theo quy ước.
- Chỉ cắt nguồn điều khiển 24 V mà quên phần động lực vẫn còn 380 V.

Tự thực hành

1. Tìm một bản vẽ tủ điện mẫu (tìm "electrical schematic sample EPLAN pdf"), in một trang sơ đồ động lực và tô màu đường đi từ aptomat tới động cơ.
2. Liệt kê mọi mã thiết bị trên trang đó và ghi tên tiếng Việt bên cạnh.
3. Nếu có đồng hồ vạn năng, đo một cục pin và một công tắc đèn đã tháo rời để quen thang V DC và thang thông mạch.

Kiểm tra nhanh

1. **Mạch điều khiển PLC trong tủ điện công nghiệp thường dùng mức điện áp nào?**

   A. 380 VAC
   B. 220 VAC
   C. 24 VDC
   D. 5 VDC

2. **Mã tham chiếu -K1 trên bản vẽ IEC thường chỉ thiết bị nào?**

   A. Động cơ
   B. Contactor hoặc relay
   C. Cảm biến
   D. Đèn báo

Đáp án: 1C, 2B

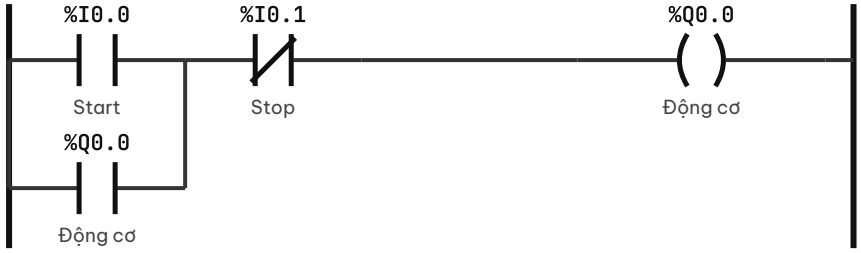
Mạch relay-contactor và mạch tự giữ

Contactor là công tắc điện từ dùng để đóng cắt tải lớn như động cơ. Nó có **cuộn hút A1–A2**, **tiếp điểm chính** 1-2, 3-4, 5-6 (cho 3 pha) và **tiếp điểm phụ**: 13-14 là thường mở (NO), 21-22 là thường đóng (NC).

Mạch tự giữ (start/stop)

Nhấn nút Start, cuộn hút có điện, contactor đóng. Tiếp điểm phụ NO của chính nó đóng lại, mắc song song với nút Start, nên khi thả tay contactor vẫn giữ. Nhấn Stop (NC) thì mạch bị cắt và contactor nhả.

Đây là mạch quan trọng nhất bạn cần thuộc, vì gần như mọi chương trình PLC đều bắt đầu từ ý tưởng này. Viết bằng ladder trên PLC nó trông như sau:



Liên động (interlock)

Khi có hai contactor không được đóng cùng lúc (đảo chiều quay, sao–tam giác), ta mắc tiếp điểm NC của contactor này nối tiếp với cuộn hút contactor kia. Ngoài khóa trong chương trình PLC, ngoài thực tế vẫn phải có **liên động bằng phần cứng** (tiếp điểm phụ hoặc khóa cơ khí), vì nếu chỉ khóa bằng phần mềm thì một contactor bị dính tiếp điểm vẫn có thể gây ngắn mạch.

Đi làm: nút Stop và nút dừng khẩn (E-Stop) luôn đấu tiếp điểm **NC** vào PLC. Lý do: nếu đứt dây, PLC đọc được mức 0 và máy dừng. Đây là nguyên tắc "an toàn khi hỏng" (fail-safe).

Đọc sơ đồ điều khiển từ trên xuống

Mạch tự giữ bằng dây gồm: L → cầu chì điều khiển → Stop (NC, 21-22) → nhánh song song gồm Start (NO, 13-14) và tiếp điểm phụ K1 (13-14) → tiếp điểm relay nhiệt (95-96, NC) → cuộn hút K1 (A1-A2) → N. Mọi phần tử "cấm chạy" đều nối tiếp, mọi phần tử "cho phép chạy" thay thế nhau thì song song. Chuyển sang PLC, quy tắc này giữ nguyên.

Ưu tiên dừng và ưu tiên chạy

Nếu nhấn Start và Stop cùng lúc thì sao? Trong mạch trên, Stop nằm nối tiếp nên luôn thắng: gọi là **ưu tiên dừng**, và là cách an toàn cho hầu hết máy. Với lệnh Set/Reset trong PLC, lệnh nào viết ở network sau sẽ thắng, nên Reset (dừng) thường được đặt sau Set.

Ví dụ thực tế

Một trạm bơm có nút Start/Stop tại tủ và thêm một cặp nút ở ngoài bể. Cách làm: các nút Start mắc **song song** với nhau, các nút Stop mắc **nối tiếp** với nhau. Thêm bao nhiêu vị trí điều khiển cũng theo đúng quy tắc đó.

Lỗi người mới hay mắc

- Quên tiếp điểm tự giữ: động cơ chỉ chạy khi giữ tay.
- Lấy nhầm tiếp điểm chính (1-2) làm tự giữ thay vì tiếp điểm phụ (13-14).
- Chỉ liên động bằng chương trình PLC, không có liên động cứng giữa hai contactor đảo chiều.
- Đặt relay nhiệt sau cuộn hút hoặc bỏ qua tiếp điểm 95-96.

Tự thực hành

1. Mở Mô phỏng, chọn bài "Khởi động / dừng động cơ", bấm "Chương trình trống" và tự viết lại không nhìn lời giải.
2. Thêm một nút Start thứ hai ở I0.2 mắc song song. Thêm nút Stop thứ hai ở I0.3 mắc nối tiếp.
3. Thử nhấn Start và Stop cùng lúc (giữ phím 0 và 1) để thấy ưu tiên dừng.

Kiểm tra nhanh

1. **Trong mạch tự giữ, tiếp điểm nào được mắc song song với nút Start?**

   A. Tiếp điểm NC của nút Stop
   B. Tiếp điểm phụ NO của chính contactor
   C. Tiếp điểm chính 1-2
   D. Relay nhiệt

2. **Vì sao nút Stop thường đấu tiếp điểm NC vào PLC?**

   A. Rẻ hơn
   B. Để PLC đọc nhanh hơn
   C. Đứt dây thì máy tự dừng (fail-safe)
   D. Vì PLC không đọc được NO

Đáp án: 1B, 2C

Cảm biến: PNP, NPN và cách đấu vào PLC

Cảm biến số (digital) chỉ có hai trạng thái: có/không có vật. Cảm biến tương tự (analog) cho giá trị liên tục như 4–20 mA, 0–10 V (nhiệt độ, áp suất, mức).

Các loại hay gặp

- **Tiệm cận điện cảm (inductive):** chỉ phát hiện kim loại, khoảng cách vài mm. Rất bền, dùng nhiều nhất.
- **Tiệm cận điện dung (capacitive):** phát hiện cả nhựa, gỗ, chất lỏng xuyên qua thành bồn nhựa.
- **Quang điện:** thu phát chung (phản xạ gương), thu phát riêng, phản xạ khuếch tán. Đếm sản phẩm trên băng tải.
- **Công tắc hành trình (limit switch):** tiếp điểm cơ khí, dùng cho giới hạn hành trình.

PNP và NPN

Cảm biến 3 dây có màu chuẩn: **nâu (BN) +24 V**, **xanh dương (BU) 0 V**, **đen (BK) tín hiệu**. Cảm biến **PNP** khi tác động sẽ xuất +24 V ra dây đen (gọi là sourcing). Cảm biến **NPN** khi tác động sẽ kéo dây đen xuống 0 V (sinking).

Ngõ vào DC của S7-1200 dùng chung một chân `1M`. Với cảm biến PNP, nối `1M` về 0 V. Với cảm biến NPN, nối `1M` lên +24 V. Không trộn hai loại trên cùng một nhóm chung.

> **Đi làm:** các máy của Châu Âu và Việt Nam chủ yếu dùng PNP, máy Nhật hay dùng NPN. Khi thay cảm biến hỏng, xem đúng mã, kiểu PNP/NPN, NO/NC và khoảng cách phát hiện. Đèn LED trên đầu cảm biến là công cụ dò lỗi đầu tiên.

Đọc nhãn một cảm biến

Ví dụ nhãn ghi: *M18, Sn 8 mm, 10–30 VDC, PNP NO, 200 mA, IP67*. Nghĩa là: thân ren 18 mm, khoảng cách phát hiện danh định 8 mm với thép (nhôm, đồng chỉ còn khoảng 30–50%), nguồn 10–30 VDC, ngõ ra PNP thường mở, tải tối đa 200 mA, chống bụi nước mức IP67. Khi lắp, đặt vật ở khoảng 70–80% Sn để ổn định.

Cảm biến 2 dây và 4 dây

Loại 2 dây mắc nối tiếp như một công tắc, có dòng rò nhỏ khi tắt nên đôi khi làm ngõ vào PLC "lửng". Loại 4 dây có cả hai ngõ ra NO và NC (dây đen và dây trắng). Cảm biến analog 2 dây 4–20 mA thì nối: +24 V → cảm biến (+) → cảm biến (–) → ngõ vào AI (+), AI (–) về 0 V.

Chống dội và chống nhiễu

Công tắc cơ khí bị dội vài mili giây khi đóng. S7-1200 có bộ lọc ngõ vào (mặc định 6,4 ms) trong Device configuration. Với tín hiệu nhanh như encoder phải giảm thời gian lọc, với nút nhấn thì giữ mặc định.

Ví dụ thực tế

Băng tải đếm chai dùng cảm biến quang thu phát riêng. Sau vài tuần máy đếm thiếu. Nguyên nhân: bụi phủ mắt thu, tín hiệu chập chờn. Người có kinh nghiệm sẽ lau mắt, kiểm tra đèn LED ổn định (stability) trên cảm biến trước, rồi mới mở laptop.

Lỗi người mới hay mắc

- Trộn PNP và NPN trên cùng một nhóm ngõ vào có chung chân 1M.
- Chọn cảm biến điện cảm để phát hiện nhựa hoặc gỗ.
- Lắp hai cảm biến tiệm cận quá sát nhau gây nhiễu lẫn nhau.
- Quên rằng cảm biến NC cho tín hiệu 1 khi không có vật, rồi viết ngược logic.

Tự thực hành

1. Vẽ ra giấy sơ đồ đấu một cảm biến PNP 3 dây vào I0.2 của CPU 1214C DC/DC/DC, ghi rõ L+, M, 1M.
2. Trong Mô phỏng, mở bài "Đếm sản phẩm trên băng tải", quan sát I0.2 bật tắt khi hộp đi qua.
3. Sửa chương trình: dùng tiếp điểm sườn lên P thay cho NO ở ngõ vào counter, xem kết quả có khác không và giải thích.

Kiểm tra nhanh

1. **Dây màu đen của cảm biến 3 dây là dây gì?**

   A. +24 V
   B. 0 V
   C. Tín hiệu ra
   D. Nối đất

2. **Đấu cảm biến PNP vào S7-1200 thì chân 1M nối vào đâu?**

   A. +24 V
   B. 0 V
   C. Để trống
   D. Chân L+ của CPU

Đáp án: 1C, 2B

Cơ cấu chấp hành: contactor, van, động cơ

PLC chỉ ra tín hiệu nhỏ. Để điều khiển tải lớn cần các khâu trung gian.

- **Ngõ ra transistor (DC/DC/DC)** của S7-1200 chịu khoảng 0,5 A mỗi kênh, đóng cắt nhanh, dùng cho cuộn relay 24 V, van điện từ, xuất xung. **Ngõ ra relay (AC/DC/RLY)** chịu tới 2 A, đóng cắt được cả AC, nhưng chậm và có tuổi thọ cơ khí.
- **Relay trung gian 24 VDC** đặt giữa PLC và cuộn contactor 220 VAC. Nó bảo vệ ngõ ra PLC và cách ly mạch.
- **Van điện từ 5/2** điều khiển xi lanh khí nén: một cuộn (đơn ổn, có lò xo trả về) hoặc hai cuộn (song ổn, nhớ trạng thái).
- **Động cơ 3 pha** khởi động trực tiếp (DOL) khi nhỏ, sao–tam giác khi lớn, hoặc qua **biến tần** khi cần thay đổi tốc độ.

Đi làm: cuộn hút relay/van DC cần diode dập (hoặc module có sẵn LED + diode). Thiếu nó, xung điện áp ngược khi ngắt có thể làm hỏng ngõ ra transistor theo thời gian.

Tính tải cho ngõ ra PLC

Một cuộn van 24 VDC công suất 4,8 W tiêu thụ I = P/U = 0,2 A, nằm trong giới hạn 0,5 A của ngõ ra transistor nên nối trực tiếp được. Cuộn contactor cỡ lớn có thể cần dòng khởi động vượt mức này, khi đó dùng relay trung gian. Cộng dòng tất cả tải 24 V để chọn bộ nguồn, ví dụ 12 van × 0,2 A + PLC 0,5 A + cảm biến 0,3 A ≈ 3,2 A, chọn nguồn 5 A.

Đọc nhãn động cơ

Nhãn ghi *Δ/Y 230/400 V, 6,1/3,5 A, 1,5 kW, cos φ 0,81, 1420 rpm, 50 Hz*. Với lưới 400 V ở Việt Nam, đấu sao và chỉnh relay nhiệt về 3,5 A. Tốc độ 1420 rpm cho biết đây là động cơ 4 cực (đồng bộ 1500 rpm).

Khí nén cơ bản

Áp suất làm việc thường 5–6 bar, qua bộ lọc – điều áp trước khi vào van. Van tiết lưu gắn trên xi lanh chỉnh tốc độ. Cảm biến từ (reed switch) kẹp trên thân xi lanh báo vị trí cuối hành trình về PLC.

Ví dụ thực tế

Máy đóng nắp có xi lanh ép. Chương trình cũ ra lệnh ép rồi chờ 1 s bằng timer. Khi áp khí yếu, xi lanh chưa ép hết đã rút về, sản phẩm lỗi. Cách sửa đúng: chờ **cảm biến hành trình cuối** tác động, và thêm timer giám sát để báo lỗi nếu quá 3 s không tới.

Lỗi người mới hay mắc

- Nối trực tiếp cuộn contactor 220 VAC vào ngõ ra transistor 24 VDC.
- Không gắn diode dập cho cuộn hút DC.
- Chỉnh relay nhiệt theo dòng tam giác trong khi động cơ đấu sao.
- Điều khiển trình tự chỉ bằng thời gian, không có cảm biến xác nhận.

Tự thực hành

1. Tính dòng tổng cho một tủ có 8 van 3 W, 6 cảm biến 20 mA, 1 PLC 0,5 A và 4 đèn báo 20 mA, rồi chọn bộ nguồn.
2. Trong Mô phỏng, mở bài "Phân loại sản phẩm", đổi thời gian đẩy TP từ 400 ms thành 100 ms và 1500 ms, xem điều gì xảy ra.
3. Tìm ảnh nhãn một động cơ 3 pha trên mạng và đọc: công suất, dòng, cách đấu, số cực.

Kiểm tra nhanh

1. **Tại sao thường dùng relay trung gian giữa PLC và contactor 220 VAC?**

   A. Để chương trình chạy nhanh
   B. Bảo vệ và cách ly ngõ ra PLC
   C. Để tiết kiệm dây
   D. Vì contactor không có cuộn hút

2. **Van 5/2 hai cuộn (song ổn) khác van một cuộn ở điểm nào?**

   A. Không dùng khí nén
   B. Nhớ vị trí khi mất tín hiệu
   C. Chỉ dùng cho nước
   D. Không cần PLC

Đáp án: 1B, 2B

PLC là gì và chu kỳ quét

PLC (Programmable Logic Controller) là máy tính công nghiệp thay thế cho hàng trăm relay. Nó gồm: **CPU**, **bộ nhớ**, **ngõ vào/ra (I/O)**, **nguồn** và **cổng truyền thông**.

Chu kỳ quét (scan cycle)

1. Đọc toàn bộ ngõ vào vật lý vào vùng nhớ ảnh **PII** (process image input).
2. Thực thi chương trình từ network đầu tới network cuối, từ trái sang phải.
3. Ghi vùng nhớ ảnh **PIQ** ra ngõ ra vật lý.
4. Xử lý truyền thông, tự chẩn đoán, rồi lặp lại.

Một chu kỳ thường chỉ vài mili giây. Hệ quả quan trọng: **thứ tự network có ý nghĩa**. Nếu cùng một cuộn dây được gán ở hai network, network sau sẽ thắng. Đây là lỗi "double coil" rất hay gặp ở người mới.

PLC có hai chế độ: **RUN** (chạy chương trình) và **STOP** (dừng, ngõ ra về 0). Bạn sẽ thấy đúng hai chế độ này trong mục Mô phỏng của app.

> **Thử ngay:** mở Mô phỏng, chuyển sang STOP rồi RUN, quan sát ngõ ra. Thời gian quét hiển thị ở thanh công cụ.

Vì sao có vùng nhớ ảnh

PLC không đọc thẳng chân ngõ vào mỗi lần gặp tiếp điểm. Nó chụp toàn bộ ngõ vào một lần ở đầu chu kỳ. Nhờ vậy trong một chu kỳ, I0.0 có cùng giá trị ở mọi network, chương trình nhất quán. Hệ quả: một xung ngắn hơn thời gian quét có thể bị bỏ lỡ, lúc đó phải dùng ngắt hoặc bộ đếm tốc độ cao.

Lần theo một chu kỳ

```
Network 1:  I0.0 ──────────( M0.0 )
Network 2:  M0.0 ──────────( Q0.0 )
Network 3:  Q0.0 ──────────( M0.1 )
```

Khi I0.0 lên 1, ngay trong cùng chu kỳ M0.0, Q0.0, M0.1 đều lên 1 vì chương trình chạy từ trên xuống. Nếu đảo thứ tự ba network, tín hiệu phải mất 3 chu kỳ mới đi tới M0.1. Trong S7-1200, mỗi chu kỳ chỉ vài mili giây nên thường không thấy, nhưng với logic sườn và bước trình tự thì thứ tự quyết định đúng sai.

Các LED trên CPU

RUN/STOP: xanh là RUN, vàng là STOP. **ERROR**: đỏ nháy là có lỗi, đỏ đứng là lỗi phần cứng. **MAINT**: vàng là có yêu cầu bảo trì. Thời gian chu kỳ tối đa mặc định là 150 ms, vượt quá thì CPU báo lỗi và có thể về STOP.

Ví dụ thực tế

Lỗi kinh điển: Q0.0 được gán bằng cuộn dây ở network 3 (chế độ tay) và network 12 (chế độ tự động). Ở chế độ tay bấm nút mà đèn không sáng, vì network 12 ghi đè 0 lên. Cách sửa: gom lại thành một network, Q0.0 = (Tay AND nút) OR (Tự động AND điều kiện).

Lỗi người mới hay mắc

- Gán một cuộn dây ở hai nơi (double coil).
- Nghĩ rằng PLC chạy mọi network "cùng lúc" như mạch điện thật.
- Dùng lệnh sườn rồi đọc lại bit đó ở network phía trên, trễ mất một chu kỳ.
- Viết vòng lặp dài trong OB1 làm vượt thời gian chu kỳ.

Tự thực hành

1. Trong Mô phỏng tạo chương trình trống với 2 network đều gán Q0.0: network 1 theo I0.0, network 2 theo I0.1. Bật I0.0 và giải thích vì sao Q0.0 không sáng.
2. Sửa lại thành một network dùng nhánh song song.
3. Chuyển RUN sang STOP khi ngõ ra đang bật, quan sát và ghi lại điều xảy ra.

Kiểm tra nhanh

1. **Một cuộn dây Q0.0 được gán ở network 2 và network 5. Ngõ ra thực tế theo network nào?**

   A. Network 2
   B. Network 5
   C. Cả hai cộng lại
   D. PLC báo lỗi và dừng

2. **Ở chế độ STOP, ngõ ra của PLC thường như thế nào?**

   A. Giữ trạng thái cũ
   B. Về 0 (tắt)
   C. Nhấp nháy
   D. Bật hết

Đáp án: 1B, 2B

Phần cứng S7-1200 và địa chỉ

Dòng S7-1200 gồm các CPU 1211C, 1212C, 1214C, 1215C, 1217C. Ở trường và trong đồ án hay dùng **CPU 1214C**: 14 ngõ vào số, 10 ngõ ra số, 2 ngõ vào analog tích hợp.

Tên biến thể cho biết **nguồn cấp / kiểu ngõ vào / kiểu ngõ ra**: `DC/DC/DC` (nguồn 24 VDC, ngõ ra transistor), `AC/DC/RLY` (nguồn 220 VAC, ngõ ra relay), `DC/DC/RLY`.

Địa chỉ

Vùng	Ví dụ	Ý nghĩa
I	`%I0.0`	Ngõ vào số: byte 0, bit 0
Q	`%Q0.0`	Ngõ ra số
M	`%M10.0`, `%MW20`	Bit/word nhớ trung gian
IW	`%IW64`	Ngõ vào analog tích hợp (kênh 0)
DB	`"Data".Speed`	Khối dữ liệu

1214C có ngõ vào `I0.0-I0.7, I1.0-I1.5` và ngõ ra `Q0.0-Q0.7, Q1.0-Q1.1`. Mỗi byte có 8 bit đánh số 0–7, nên không có `I0.8`.

Kiểu dữ liệu cơ bản

`Bool` (1 bit), `Byte` (8 bit), `Word` / `Int` (16 bit), `DWord` / `DInt` / `Real` (32 bit), `Time` (ví dụ `T#5s`), `String`.

Mẹo có thật trong TIA Portal: trong thuộc tính CPU, bật *System and clock memory*. Nếu đặt clock memory ở MB0 thì `M0.5` nhấp nháy 1 Hz; system memory ở MB1 cho `M1.0` = FirstScan, `M1.2` = AlwaysTRUE.

So sánh các CPU

CPU	DI / DO tích hợp	AI / AO	Module mở rộng
1211C	6 / 4	2 / 0	Không gắn được SM
1212C	8 / 6	2 / 0	2 SM
1214C	14 / 10	2 / 0	8 SM
1215C	14 / 10	2 / 2	8 SM, 2 cổng Ethernet

Module mở rộng: **SM** (tín hiệu) gắn bên phải, ví dụ SM 1221 (DI), SM 1222 (DQ), SM 1231 (AI), SM 1232 (AQ). **CM** (truyền thông) gắn bên trái, tối đa 3, ví dụ CM 1241 RS485. **SB** (signal board) cắm ngay mặt trước CPU, thêm vài I/O mà không tốn chỗ.

Byte, word và bẫy chồng địa chỉ

`MW10` gồm hai byte MB10 và MB11. `MW11` gồm MB11 và MB12, tức là chồng lên MW10. Vì vậy word đặt cách nhau 2 (MW10, MW12…), double word cách nhau 4 (MD20, MD24…). Bit M10.0 cũng nằm trong MW10. Cách tránh hẳn lỗi này là dùng DB với tên biến thay cho vùng M.

Mã đặt hàng

Dạng `6ES7 214-1AG40-0XB0` là 1214C DC/DC/DC, `…1BG40…` là AC/DC/RLY, `…1HG40…` là DC/DC/RLY. Khi thêm thiết bị trong TIA Portal phải chọn đúng mã và đúng phiên bản firmware, sai thì không download được.

Ví dụ thực tế

Đồ án băng tải phân loại cần: 2 nút, 1 E-Stop, 4 cảm biến, 1 encoder (2 ngõ vào), tổng 9 DI; 1 động cơ, 2 van, 3 đèn, tổng 6 DO. CPU 1214C DC/DC/DC đủ dùng và còn dư để mở rộng, ngõ ra transistor phù hợp với van 24 V.

Lỗi người mới hay mắc

- Dùng MW10 và MW11 cho hai biến khác nhau.
- Chọn bản relay (RLY) rồi muốn phát xung tốc độ cao cho động cơ bước.
- Quên bật System and clock memory rồi dùng M0.5, M1.0 như có sẵn.
- Chọn sai firmware khi Add new device.

Tự thực hành

1. Lập bảng I/O cho một máy bạn tự nghĩ ra (ít nhất 6 ngõ vào, 4 ngõ ra) gồm cột: địa chỉ, tên tag, mô tả, loại thiết bị.
2. Chọn CPU phù hợp cho bảng đó và giải thích.
3. Viết ra MD100 gồm những byte nào, và word nào không được dùng chung với nó.

Kiểm tra nhanh

1. **Địa chỉ nào sau đây KHÔNG hợp lệ?**

   A. %I0.7
   B. %I1.0
   C. %I0.8
   D. %Q0.0

2. **CPU 1214C DC/DC/DC có ngõ ra kiểu gì?**

   A. Relay
   B. Transistor
   C. Triac
   D. Analog

Đáp án: 1C, 2B

TIA Portal và PLCSIM: quy trình làm một project

1. **Create new project**, đặt tên có ngày tháng, lưu vào ổ không đồng bộ đám mây (tránh lỗi khóa file).
2. **Add new device**: chọn đúng mã CPU (order number) và đúng firmware. Mã in trên mặt CPU, ví dụ dạng `6ES7 214-1AG40-0XB0`.
3. **Device configuration**: đặt địa chỉ IP (ví dụ 192.168.0.1), bật clock memory, thêm module mở rộng.
4. **PLC tags**: khai báo tên cho mọi địa chỉ, ví dụ `Start_PB → %I0.0`. Không lập trình bằng địa chỉ trần.
5. **Program blocks → Main [OB1]**: viết ladder.
6. **Compile** (biên dịch) và sửa hết lỗi.
7. **Start simulation** (PLCSIM) hoặc **Download to device**, rồi **Monitoring on** (nút kính đeo) để xem dòng điện chạy trong ladder.
8. Dùng **Watch table** và **Force table** để thử tín hiệu.

Trên PLCSIM bạn chỉ thay đổi được ngõ vào bằng tay. Muốn có "nhà máy ảo" để chạy cùng, các bạn sinh viên hay dùng **Factory I/O** kết nối với PLCSIM Advanced hoặc qua driver phù hợp.

> **Đi làm:** luôn **upload/so sánh (Compare online/offline)** trước khi sửa chương trình đang chạy trong máy của khách hàng, và lưu bản sao lưu có ghi ngày. Mất chương trình gốc là lỗi không được phép mắc.

Các công cụ phải thạo

Công cụ	Dùng để
Monitoring on/off	Xem dòng chảy trong ladder: xanh liền là có, xanh đứt là không.
Watch table	Theo dõi và sửa giá trị biến một lần (Modify).
Force table	Ép cứng ngõ vào/ra bất kể chương trình. Nhớ tắt force khi xong.
Cross-reference	Tìm mọi nơi một tag được đọc hoặc ghi. Công cụ số một để tìm double coil.
Online & diagnostics	Đọc bộ đệm chẩn đoán, biết vì sao CPU về STOP.
Compare offline/online	Biết chương trình trên máy tính có giống trong PLC không.

Download và mạng

Máy tính và PLC phải cùng dải mạng: PLC 192.168.0.1 thì đặt card mạng máy tính 192.168.0.x, mặt nạ 255.255.255.0. PLC mới chưa có IP thì dùng *Accessible devices* để tìm và gán. Nếu chỉ sửa logic thì download phần mềm, nếu đổi cấu hình phần cứng thì CPU phải về STOP.

Lỗi compile thường gặp

"Tag not defined": gõ sai tên hoặc chưa khai báo. "Data type mismatch": hai vế khác kiểu, ví dụ Int với Real. Dấu hỏi đỏ `<???>` trên lệnh: chưa điền toán hạng. Timer chưa có instance DB: bấm OK khi TIA Portal hỏi tạo DB.

Ví dụ thực tế

Khách gọi báo máy dừng. Bạn kết nối, mở Online & diagnostics, đọc Diagnostics buffer thấy dòng "Cycle time exceeded". Từ đó biết phải tìm vòng lặp hoặc khối chạy quá lâu, thay vì dò từng cảm biến.

Lỗi người mới hay mắc

- Lưu project trong thư mục đồng bộ đám mây, bị khóa file và hỏng project.
- Để quên Force đang bật khi rời máy.
- Download đè mà không upload và sao lưu bản đang chạy.
- Đặt IP máy tính khác dải với PLC rồi tưởng cáp hỏng.

Tự thực hành

1. Cài TIA Portal bản dùng thử, tạo project CPU 1214C DC/DC/DC, bật clock memory.
2. Khai báo 4 tag, viết mạch tự giữ, compile không lỗi, chạy PLCSIM và bật Monitoring.
3. Tạo Watch table với 4 tag đó, dùng Modify để bật Start. Sau đó mở Cross-reference của tag ngõ ra.

Kiểm tra nhanh

1. **Vì sao nên khai báo PLC tags thay vì dùng địa chỉ trần?**

   A. Bắt buộc mới compile được
   B. Chương trình dễ đọc, dễ bảo trì
   C. Chạy nhanh hơn
   D. Tiết kiệm bộ nhớ

2. **Trước khi sửa chương trình đang chạy ở nhà máy, việc đầu tiên nên làm?**

   A. Download đè luôn
   B. Xóa chương trình cũ
   C. Upload và lưu bản sao lưu, so sánh online/offline
   D. Tắt nguồn PLC

Đáp án: 1B, 2C

Lệnh bit: NO, NC, cuộn dây, Set/Reset, sườn

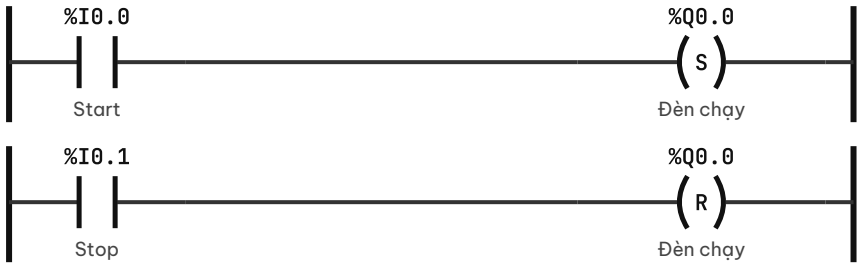
Ký hiệu	Tên	Dẫn khi
`--| |--`	Tiếp điểm thường mở (NO)	Bit = 1
`--|/|--`	Tiếp điểm thường đóng (NC)	Bit = 0
`--|P|--`	Sườn lên	Đúng 1 chu kỳ khi bit chuyển 0→1
`--|N|--`	Sườn xuống	Đúng 1 chu kỳ khi bit chuyển 1→0
`--( )`	Cuộn dây	Bit = kết quả logic
`--(S)` / `--(R)`	Set / Reset	Nhớ 1 / xóa về 0

Nối tiếp là phép **AND**, song song là phép **OR**. Tiếp điểm NC trong chương trình là phép **NOT**.

Bẫy hay gặp: NC trong chương trình và NC ngoài thực tế

Nếu nút Stop đấu NC ngoài thực tế thì khi *không nhấn*, I0.1 = 1. Trong chương trình, bạn phải dùng tiếp điểm **NO** của I0.1 để máy chạy được. Bài tập "Đảo chiều động cơ" trong app mô phỏng đúng tình huống này.

Ví dụ dùng Set/Reset thay cho tự giữ:



Bảng trạng thái cho mạch tự giữ

Start	Stop	Q trước đó	Q mới
0	0	0	0
1	0	0	1
0	0	1	1 (tự giữ)
x	1	x	0 (ưu tiên dừng)

Viết thành biểu thức: `Q = (Start OR Q) AND NOT Stop`. Mọi network ladder đều đọc được thành biểu thức logic như vậy, và ngược lại.

Khi nào dùng cuộn dây, khi nào dùng Set/Reset

Cuộn dây thường `-( )-` được ghi lại mỗi chu kỳ, mất điều kiện là tắt, dễ đọc và an toàn. Set/Reset nhớ trạng thái, hợp với bước trình tự và cảnh báo cần xác nhận, nhưng phải chắc chắn có đường Reset trong mọi tình huống kể cả lỗi và dừng khẩn. Trong S7-1200 còn có khối **SR** (ưu tiên Reset) và **RS** (ưu tiên Set).

Sườn dùng để làm gì

Một nút nhấn vừa bật vừa tắt đèn (toggle): dùng sườn lên của nút, nếu không thì giữ tay một giây đèn đã đảo hàng trăm lần. Counter và các lệnh chỉ làm một lần (ghi nhật ký, cộng dồn) cũng cần sườn.

Ví dụ thực tế

Đèn báo lỗi phải sáng mãi cho tới khi người vận hành xác nhận, dù lỗi đã hết. Cách viết: sườn lên của tín hiệu lỗi → Set `Alarm`. Nút Ack AND NOT lỗi → Reset `Alarm`.

Lỗi người mới hay mắc

- Dùng tiếp điểm NC trong chương trình cho nút Stop đã đấu NC ngoài thực tế, máy không bao giờ chạy.
- Set một bit mà không có network nào Reset.
- Dùng cùng một bit nhớ sườn cho hai lệnh sườn khác nhau trong TIA Portal.
- Quên rằng nối tiếp là AND, đặt điều kiện "một trong hai" thành nối tiếp.

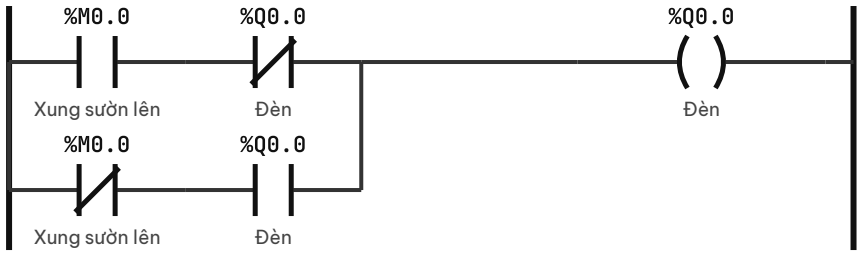
Tự thực hành

1. Trong Mô phỏng, viết mạch một nút bật tắt đèn: dùng tiếp điểm P của I0.0, kết hợp Q0.0 và bit nhớ M0.0. Gợi ý: hai network Set và Reset với điều kiện ngược nhau, dùng M0.0 làm xung sườn.
2. Viết mạch cảnh báo có xác nhận như ví dụ trên với I0.0 là lỗi, I0.1 là Ack.
3. Viết bảng trạng thái cho mạch đảo chiều có liên động.

Bài giải mẫu: một nút vừa bật vừa tắt

Yêu cầu: mỗi lần nhấn I0.0, đèn Q0.0 đảo trạng thái. Lời giải gồm hai network:





Network 1 tạo xung M0.0 dài đúng một chu kỳ ở sườn lên của nút. Network 2 là phép XOR: nếu có xung và đèn đang tắt thì bật (nhánh trên), nếu không có xung và đèn đang sáng thì giữ (nhánh dưới). Khi có xung mà đèn đang sáng, cả hai nhánh đều hở nên đèn tắt.

Xung M0.0	Q0.0 trước	Q0.0 sau
0	0	0
0	1	1 (giữ)
1	0	1 (bật)
1	1	0 (tắt)

Thử bỏ lệnh sườn, thay bằng tiếp điểm NO thường: giữ nút là đèn đảo liên tục mỗi chu kỳ, mắt thấy đèn sáng mờ và trạng thái cuối cùng là ngẫu nhiên. Đó là lý do phải có sườn.

Kiểm tra nhanh

1. **Hai tiếp điểm mắc nối tiếp tương đương phép logic nào?**
   A. OR
   B. AND
   C. XOR
   D. NOT

2. **Nút Stop đấu NC ngoài thực tế. Trong ladder nên dùng tiếp điểm gì cho Stop?**
   A. NC
   B. NO
   C. Sườn lên
   D. Không cần

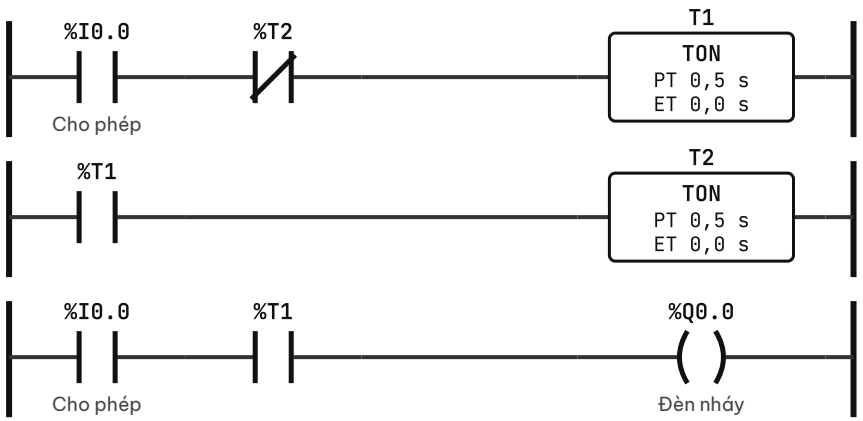
Đáp án: 1B, 2B

Timer IEC: TON, TOF, TP, TONR

Trong S7-1200, timer là khối IEC có instance DB riêng. Ngõ vào `IN`, thời gian đặt `PT` (kiểu Time, ví dụ `T#5s`), ngõ ra `Q` và thời gian đã chạy `ET`.

- **TON (on-delay):** IN = 1 liên tục đủ PT thì Q = 1. IN về 0 thì reset ngay. Dùng cho trễ khởi động, chống dội, kiểm tra cảm biến kẹt.
- **TOF (off-delay):** IN = 1 thì Q = 1 ngay. IN về 0 thì Q giữ thêm PT rồi mới tắt. Dùng cho quạt làm mát chạy thêm sau khi dừng máy, đèn cầu thang.
- **TP (pulse):** sườn lên của IN tạo xung Q dài đúng PT. Dùng cho xi lanh đẩy, còi báo ngắn.
- **TONR:** giống TON nhưng cộng dồn thời gian, phải reset bằng chân R. Dùng đếm giờ chạy máy để bảo trì.

Mạch tạo xung nhấp nháy bằng 2 TON:



Trong app: PT được nhập bằng mili giây (3000 = 3 s). Có thể so sánh `T1.ET` bằng tiếp điểm so sánh để chia nhiều giai đoạn chỉ với một timer.

Giản đồ thời gian

```
IN   : ____██████████____██____
TON  : ________███████__________   (trễ bật, PT = 4 ô)
TOF  : ____██████████████______   (trễ tắt, PT = 4 ô)
TP   : ____████____________████__   (xung cố định, PT = 4 ô)
```

Xung IN thứ hai ngắn hơn PT: TON không kịp bật, TP vẫn phát đủ xung, TOF kéo dài.

Trong TIA Portal

Kéo lệnh TON vào, TIA Portal hỏi tạo **instance DB** (ví dụ `IEC_Timer_0_DB`). Giá trị thời gian viết `T#500ms`, `T#5s`, `T#1m30s`. Đọc kết quả ở nơi khác bằng `"IEC_Timer_0_DB".Q` và `.ET`. Khi viết trong FB, chọn *Multi-instance* để timer nằm gọn trong DB của FB đó.

Ứng dụng hay gặp

- **Chống dội / lọc nhiễu:** cảm biến phải bật liên tục 200 ms mới tính (TON).
- **Giám sát (watchdog):** ra lệnh mở van, sau 5 s chưa có tín hiệu "đã mở" thì báo lỗi (TON).
- **Chạy thêm:** quạt chạy thêm 60 s sau khi tắt gia nhiệt (TOF).
- **Đếm giờ chạy:** TONR cộng dồn mỗi khi động cơ chạy, đủ giờ thì nhắc bảo trì.

Ví dụ thực tế

Bơm định lượng: nhấn nút, bơm chạy đúng 3 s dù người dùng giữ hay nhả tay. Đây là việc của TP. Nếu dùng TON sẽ phải thêm mạch tự giữ và phức tạp hơn.

Lỗi người mới hay mắc

- Dùng một instance DB cho hai timer khác nhau, hai timer giẫm lên nhau.
- Gọi timer có điều kiện (trong nhánh không phải lúc nào cũng chạy), timer không cập nhật.
- Nhập PT là số 5 thay vì T#5s.
- Quên rằng TON reset ngay khi IN mất, rồi thắc mắc vì sao thời gian không cộng dồn.

Tự thực hành

1. Trong Mô phỏng, làm bài "Đèn cầu thang hẹn giờ" và "Đèn báo lỗi nhấp nháy" từ chương trình trống.
2. Thêm một ngõ ra kiểu TONR, bật tắt ngõ vào nhiều lần và quan sát ET cộng dồn, sau đó Reset nó.
3. Viết mạch giám sát: I0.0 là lệnh mở van, I0.1 là tín hiệu đã mở, Q0.1 báo lỗi nếu sau 3 s có lệnh mà chưa có tín hiệu.

Kiểm tra nhanh

1. **Quạt làm mát cần chạy thêm 2 phút sau khi máy dừng. Dùng timer nào?**

   A. TON
   B. TOF
   C. TP
   D. CTU

2. **Với TON, nếu IN tắt trước khi đủ PT thì sao?**

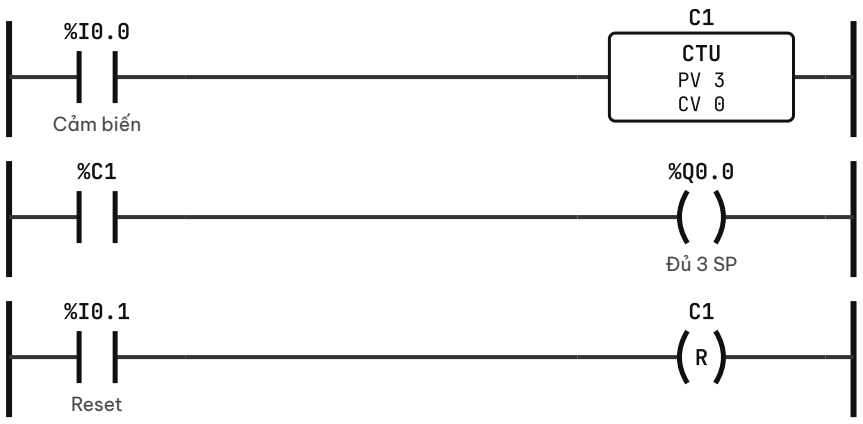
   A. ET giữ nguyên
   B. ET reset về 0
   C. Q vẫn lên 1
   D. Timer báo lỗi

Đáp án: 1B, 2B

Counter, so sánh, MOVE và toán học

CTU đếm lên mỗi sườn lên của CU, `Q = 1` khi `CV ≥ PV`, chân `R` xóa về 0. **CTD** đếm xuống, chân `LD` nạp giá trị PV. **CTUD** đếm cả hai chiều.

Lệnh **so sánh** (`=, <>, >, ≥, <, ≤`) là một tiếp điểm, đóng khi điều kiện đúng. **MOVE** chép giá trị. **ADD, SUB, MUL, DIV** làm toán, chú ý kiểu dữ liệu hai vế phải giống nhau (Int với Int, Real với Real), nếu không phải dùng **CONVERT**.



Đi làm: counter thường dùng để đếm sản lượng ca, đếm số lần đóng cắt để báo bảo trì. Giá trị cần giữ khi mất điện thì đặt **Retain** trong DB.

Kiểu dữ liệu và tràn số

`Int` chứa từ −32768 đến 32767, `DInt` tới khoảng ±2,1 tỷ, `Real` là số thực 32 bit với khoảng 6–7 chữ số có nghĩa. Bộ đếm sản lượng cả năm nên dùng DInt. Phép chia hai số Int cho kết quả Int: 7 / 2 = 3. Muốn 3,5 phải đổi sang Real trước.

Ví dụ tính tốc độ dây chuyền

```
Mỗi 6 giây (xung từ timer):
  SUB   Count_Now - Count_Last → Delta
  MUL   Delta * 10             → Speed_ppm   (sản phẩm/phút)
  MOVE  Count_Now              → Count_Last
```

CTD và CTUD

CTD: nạp PV bằng chân LD, mỗi sườn CD giảm 1, Q = 1 khi CV ≤ 0. Dùng cho "còn bao nhiêu sản phẩm nữa thì đầy thùng". **CTUD**: đếm xe vào ra bãi, QU báo đầy, QD báo trống.

Bộ đếm tốc độ cao (HSC)

Counter thường chỉ đếm được xung chậm hơn chu kỳ quét. Encoder vài kHz phải dùng HSC cấu hình trong Device configuration, đọc giá trị ở địa chỉ như `ID1000`.

Ví dụ thực tế

Máy đóng thùng 12 chai: cảm biến đếm chai bằng CTU với PV = 12. Khi Q lên: dừng băng chai, ra lệnh đẩy thùng, đồng thời cộng 1 vào bộ đếm thùng của ca, rồi Reset CTU. Bộ đếm ca đặt Retain để không mất khi cúp điện.

Lỗi người mới hay mắc

- Đưa thẳng tín hiệu dội (nút nhấn cơ khí) vào counter, một lần nhấn đếm thành ba.
- Reset counter bằng chính ngõ Q của nó ở cùng network, tạo xung chỉ dài một chu kỳ mà nơi khác không kịp đọc.
- Chia hai số Int rồi thắc mắc vì sao mất phần lẻ.
- Dùng counter thường cho encoder tốc độ cao.

Tự thực hành

1. Trong Mô phỏng, mở bài "Đếm sản phẩm", đổi PV thành 3 và thêm network: khi đủ số lượng, sau 2 s tự Reset counter để chạy mẻ tiếp theo.
2. Thêm một counter kiểu CTD nạp 5, mỗi hộp đi qua giảm 1, đèn sáng khi về 0.
3. Viết ra giấy chuỗi lệnh tính phần trăm sản phẩm đạt = đạt / tổng × 100 với kiểu dữ liệu đúng.

Kiểm tra nhanh

1. **CTU tăng CV khi nào?**

   A. Mỗi chu kỳ quét khi CU = 1
   B. Mỗi sườn lên của CU
   C. Mỗi giây
   D. Khi R = 1

2. **Cộng một biến Int với một biến Real, bạn cần làm gì?**

   A. Cộng trực tiếp
   B. Dùng CONVERT cho cùng kiểu
   C. Dùng counter
   D. Không thể làm được

Đáp án: 1B, 2B

Lập trình trình tự theo bước (step)

Đa số máy chạy theo trình tự: nạp liệu → trộn → xả. Cách viết dễ đọc và dễ dò lỗi nhất là chia thành các **bước**, mỗi bước là một bit nhớ (hoặc một số nguyên `Step`).

1. Vẽ **lưu đồ hoặc GRAFCET**: mỗi bước làm gì, điều kiện chuyển bước là gì.
2. Mỗi network chuyển bước: *bước hiện tại AND điều kiện* → Reset bước hiện tại, Set bước kế tiếp.
3. Network ngõ ra riêng: ngõ ra = OR các bước cần nó. Không Set/Reset ngõ ra rải rác khắp nơi.

Cách dùng số nguyên: `MOVE 10 → Step`, rồi ở mỗi network so sánh `Step == 10`. Đánh số cách nhau 10 để sau này chèn bước 15 mà không phải đánh lại.

Thực hành: làm bài "Trộn hóa chất theo trình tự" trong mục Bài tập. Khi máy dừng giữa chừng, nhìn bit bước nào đang bật là biết máy kẹt ở đâu.

Ví dụ đầy đủ: máy khoan tự động

Bước	Hành động	Điều kiện chuyển
0 Chờ	Không	Start AND có phôi AND máy ở vị trí gốc
10 Kẹp	Van kẹp	Cảm biến đã kẹp
20 Khoan xuống	Van kẹp, động cơ khoan, van xuống	Hành trình dưới
30 Rút lên	Van kẹp, động cơ khoan	Hành trình trên
40 Nhả kẹp	Không	Cảm biến đã nhả, về bước 0

Network ngõ ra: `Van kẹp = Step10 OR Step20 OR Step30`. Mỗi ngõ ra chỉ xuất hiện một lần trong chương trình.

Những thứ máy thật luôn có thêm

- **Giám sát thời gian từng bước:** ở một bước quá lâu thì báo lỗi kèm số bước.
- **Auto / Manual:** ở Manual, trình tự dừng và từng cơ cấu chạy bằng nút riêng, vẫn giữ liên động an toàn.
- **Dừng cuối chu kỳ** khác **dừng ngay**. E-Stop thì xóa mọi bước và cắt ngõ ra.
- **Về gốc (home):** sau khi mất điện phải đưa cơ cấu về vị trí ban đầu trước khi cho chạy tự động.

Trong S7-1500 có ngôn ngữ GRAPH vẽ trực tiếp sơ đồ bước. Với S7-1200 không có GRAPH, nên cách dùng bit bước hoặc biến số nguyên là kỹ năng bắt buộc.

Ví dụ thực tế

Máy dừng giữa chừng lúc 2 giờ sáng. Người vận hành đọc trên HMI: "Bước 20 quá thời gian". Thợ bảo trì đi thẳng tới hành trình dưới của đầu khoan và thấy cảm biến lệch. Không cần mở laptop. Đó là giá trị của lập trình theo bước có giám sát.

Lỗi người mới hay mắc

- Set bước mới mà quên Reset bước cũ, hai bước cùng hoạt động.
- Viết network chuyển bước theo thứ tự làm máy nhảy liền hai bước trong một chu kỳ khi điều kiện kế tiếp đã sẵn đúng.
- Set/Reset ngõ ra rải rác trong từng bước thay vì gom về một network.
- Không có đường thoát khi cảm biến hỏng: máy treo mãi ở một bước.

Tự thực hành

1. Trong Mô phỏng, mở bài "Trộn hóa chất theo trình tự" và đọc kỹ từng network.
2. Thêm bước thứ tư: sau khi xả, đèn báo Q0.3 sáng 2 s rồi mới về chờ.
3. Thêm nút dừng I0.1: nhấn là Reset mọi bước ngay lập tức.

Kiểm tra nhanh

1. **Vì sao nên đánh số bước 10, 20, 30 thay vì 1, 2, 3?**

   A. PLC yêu cầu
   B. Dễ chèn bước mới mà không đánh số lại
   C. Chạy nhanh hơn
   D. Tiết kiệm bộ nhớ

2. **Điều kiện chuyển từ bước A sang bước B thường gồm?**

   A. Chỉ cần điều kiện
   B. Bước A đang hoạt động AND điều kiện chuyển
   C. Bước B đang hoạt động
   D. Nút Start

Đáp án: 1B, 2B

Lập trình cấu trúc: OB, FC, FB, DB

Khối	Vai trò
OB	Do hệ điều hành gọi. `OB1 Main` chạy liên tục, `OB100 Startup` chạy một lần khi khởi động, OB Cyclic interrupt chạy theo chu kỳ cố định (dùng cho PID).
FC	Hàm không có bộ nhớ riêng. Dùng cho tính toán, chuyển đổi.
FB	Khối có bộ nhớ (instance DB). Dùng khi một đối tượng cần nhớ trạng thái: một động cơ, một van, một băng tải.
DB	Khối dữ liệu toàn cục hoặc instance.

Ví dụ thực tế: viết một FB `FB_Motor` có Start, Stop, Feedback, Fault, rồi gọi 10 lần cho 10 động cơ với 10 instance DB. Sửa logic một chỗ, cả 10 động cơ được cập nhật.

Ngôn ngữ: **LAD** (ladder) để dò lỗi trực quan, **FBD**, và **SCL** (giống Pascal) cho tính toán, vòng lặp, xử lý mảng. Nhà tuyển dụng đánh giá cao ứng viên biết cả LAD và SCL.

Quy ước tên: chọn một kiểu và giữ nhất quán, ví dụ `M01_Conveyor_Run`, `V03_Valve_Open`. Viết comment cho mọi network.

Giao diện của một khối

Vùng	Ý nghĩa
Input	Giá trị đưa vào khối, chỉ đọc.
Output	Kết quả khối trả ra.
InOut	Vừa đọc vừa ghi, dùng cho cấu trúc dữ liệu lớn.
Static	Chỉ có ở FB, nhớ qua các chu kỳ (lưu trong instance DB).
Temp	Biến tạm, mất sau khi khối chạy xong. Phải ghi trước khi đọc.

Ví dụ FB_Motor

```
Input : Cmd_Start, Cmd_Stop, Feedback, Overload, Reset
Output: Run, Fault
Static: FbkTimer (TON, multi-instance)

Run   := (Cmd_Start OR Run) AND NOT Cmd_Stop AND NOT Fault
FbkTimer(IN := Run AND NOT Feedback, PT := T#2s)
Fault := (FbkTimer.Q OR Overload OR Fault) AND NOT Reset
```

Gọi trong OB1: `"M01_DB"(Cmd_Start := "M01_Start", …, Run ⇒ "M01_KM")`. Thêm động cơ thứ hai chỉ là thêm một lần gọi với DB khác.

UDT và cách tổ chức project

UDT (kiểu dữ liệu tự định nghĩa) gom dữ liệu một thiết bị, ví dụ `typeMotor` gồm Run, Fault, Hours. Thư mục chương trình nên chia theo khu vực máy: 10_Nạp liệu, 20_Trộn, 30_Đóng gói, 90_Cảnh báo. OB1 chỉ gọi các FC khu vực, không chứa logic chi tiết.

Ví dụ thực tế

Nhà máy có 24 băng tải. Khách yêu cầu thêm thời gian trễ 1 s trước khi khởi động kèm còi báo. Với FB_Conveyor, bạn sửa một chỗ và download. Nếu 24 băng tải viết tay riêng lẻ, đó là 24 chỗ sửa và 24 cơ hội sai.

Lỗi người mới hay mắc

- Dùng FC cho thứ cần nhớ trạng thái (tự giữ, timer) rồi thấy nó chạy sai.
- Đọc biến Temp trước khi ghi, nhận giá trị rác.
- Dùng tag toàn cục trực tiếp bên trong FB, khối không tái sử dụng được.
- Dùng chung một instance DB cho hai lần gọi FB.

Tự thực hành

1. Trong TIA Portal tạo FB_Motor theo ví dụ trên bằng LAD, gọi 2 lần với 2 instance DB.
2. Chạy PLCSIM, cho Feedback = 0 và kiểm tra Fault bật sau 2 s.
3. Tạo UDT typeMotor và một DB toàn cục chứa mảng 4 phần tử kiểu đó.

Kiểm tra nhanh

1. **Bạn cần điều khiển 8 động cơ giống nhau, mỗi cái nhớ trạng thái riêng. Dùng khối nào?**

   A. Một FC duy nhất
   B. Một FB, gọi 8 lần với 8 instance DB
   C. 8 OB
   D. Chỉ cần DB

2. **OB nào chạy một lần khi PLC chuyển từ STOP sang RUN?**

   A. OB1
   B. OB100
   C. OB30
   D. OB82

Đáp án: 1B, 2B

Ngôn ngữ SCL: viết PLC bằng câu lệnh

SCL (Structured Control Language) là ngôn ngữ dạng văn bản theo chuẩn IEC 61131-3, ở hãng khác gọi là ST (Structured Text). Cú pháp giống Pascal. Ladder mạnh ở logic đóng cắt và dễ dò lỗi bằng mắt, còn SCL mạnh ở **tính toán, so sánh nhiều nhánh, vòng lặp và xử lý mảng**. Một kỹ sư làm việc thật dùng cả hai: LAD cho liên động, SCL cho công thức và xử lý dữ liệu.

Tạo khối SCL

Trong TIA Portal: *Add new block* → chọn FC hoặc FB → mục Language chọn **SCL**. Các bản TIA Portal mới còn cho chèn một network SCL vào giữa khối LAD (chuột phải vào network → Insert SCL network), rất tiện khi chỉ cần vài dòng tính toán.

Cú pháp nền tảng

Việc	Cách viết	Ghi chú
Gán giá trị	`#Speed := 50.0;`	Dùng `:=`, kết thúc bằng `;`
Tag toàn cục	`"Motor_Run"`	Trong ngoặc kép
Biến cục bộ của khối	`#Count`	Có dấu # phía trước
Biến trong DB	`"Data".Level`	Tên DB, dấu chấm, tên biến
So sánh	`=  <>  >  >=  <  <=`	Bằng là một dấu `=`
Logic	`AND  OR  NOT  XOR`	
Chú thích	`// một dòng` hoặc `(* nhiều dòng *)`	

Từ ladder sang SCL

Mạch tự giữ quen thuộc viết bằng một dòng:

```
"Motor" := ("Start" OR "Motor") AND NOT "Stop";
```

Đây chính là biểu thức logic của network ladder. Viết kiểu gán trực tiếp như trên giữ đúng tính chất của cuộn dây: mất điều kiện là tắt.

IF và CASE

```
IF #Level ⩾ 80.0 THEN
    #Pump := FALSE;
ELSIF #Level ⩽ 20.0 THEN
    #Pump := TRUE;
END_IF;   // giữa 20 và 80: giữ nguyên trạng thái cũ
```

Chú ý: biến chỉ được gán trong nhánh IF sẽ **giữ giá trị cũ** khi không nhánh nào đúng, tức là hoạt động như Set/Reset. Điều này hữu ích ở ví dụ bơm có trễ ở trên, nhưng cũng là nguồn lỗi nếu bạn quên nhánh ELSE.

```
CASE #Step OF
    0:  // Chờ
        IF #Start THEN #Step := 10; END_IF;
    10: // Nạp
        #ValveIn := TRUE;
        IF #LevelHigh THEN #ValveIn := FALSE; #Step := 20; END_IF;
    20: // Khuấy
        #Mixer := TRUE;
        IF #MixDone THEN #Mixer := FALSE; #Step := 0; END_IF;
    ELSE
        #Step := 0;
END_CASE;
```

CASE với biến bước là cách viết máy trạng thái gọn nhất trong S7-1200, thay cho GRAPH vốn chỉ có ở S7-1500.

Vòng lặp và mảng

```
#Sum := 0.0;
FOR #i := 0 TO 9 DO
    #Sum := #Sum + #Samples[#i];
END_FOR;
#Average := #Sum / 10.0;
```

Vòng lặp chạy trọn trong một chu kỳ quét. Vòng lặp vài chục phần tử không sao, nhưng vòng lặp hàng chục nghìn lần hoặc WHILE không có điều kiện thoát sẽ làm vượt thời gian chu kỳ và CPU về STOP.

Gọi timer và khối khác

```
#DelayTimer(IN := #Run AND NOT #Feedback,
            PT := T#2s);
#Fault := #DelayTimer.Q;
```

`#DelayTimer` khai báo trong vùng Static của FB với kiểu `TON_TIME` (multi-instance). Tham số vào dùng `:=`, tham số ra dùng `⇒`.

Chuyển đổi kiểu dữ liệu

SCL kiểm tra kiểu chặt hơn bạn nghĩ. Dùng hàm chuyển đổi rõ ràng: `INT_TO_REAL(#Raw)`, `REAL_TO_INT(#Value)` (làm tròn), `TRUNC(#Value)` (cắt phần lẻ). Hằng số thực viết có dấu chấm: `10.0` chứ không phải `10`.

> **Đi làm:** nhiều công ty quy định LAD cho phần liên động và an toàn để thợ bảo trì đọc được, SCL cho tính toán, công thức, truyền thông. Hồ sơ có một FB viết bằng SCL chạy đúng là điểm cộng rõ ràng khi phỏng vấn.

Bài mẫu hoàn chỉnh: FC đổi analog sang đơn vị kỹ thuật

```
FUNCTION "FC_Scale" : Real
VAR_INPUT
    Raw    : Int;    // giá trị đọc từ %IW
    RawMin : Int;    // 5530 với 4-20 mA, 0 với 0-10 V
    EngMin : Real;   // đầu dải, ví dụ 0.0 bar
    EngMax : Real;   // cuối dải, ví dụ 10.0 bar
END_VAR
VAR_TEMP
    norm : Real;
END_VAR

#norm := (INT_TO_REAL(#Raw) - INT_TO_REAL(#RawMin))
         / (27648.0 - INT_TO_REAL(#RawMin));
IF #norm < 0.0 THEN #norm := 0.0; END_IF;
IF #norm > 1.0 THEN #norm := 1.0; END_IF;
"FC_Scale" := #EngMin + #norm * (#EngMax - #EngMin);
```

Thử bằng số: Raw = 16589, RawMin = 5530, dải 0–10 bar. norm = 11059 / 22118 = 0,5. Kết quả 5,0 bar. Hai dòng IF giới hạn giúp HMI không hiện số âm khi cảm biến hơi lệch dưới 4 mA.

Ví dụ thực tế

Máy đóng gói có 6 công thức sản phẩm, mỗi công thức gồm thời gian hàn, nhiệt độ, tốc độ. Viết bằng ladder cần 18 lệnh MOVE trong 6 network. Viết bằng SCL là một lệnh CASE theo số công thức, hoặc gọn hơn nữa là một mảng cấu trúc và một dòng gán `#Active := "Recipes".Item[#Index];`.

Lỗi người mới hay mắc

- Viết = thay cho := khi gán. Compiler sẽ báo lỗi, đọc kỹ thông báo.

- Quên dấu ; cuối câu hoặc quên END_IF, END_CASE, END_FOR.
- Viết 10 thay vì 10.0 trong phép tính Real, hoặc chia hai số Int và mất phần lẻ.
- Gán ngõ ra trong IF mà không có ELSE, ngõ ra bị kẹt ở 1.
- Viết WHILE chờ một cảm biến: vòng lặp không thoát trong một chu kỳ, CPU về STOP. Chờ tín hiệu phải dùng bước (CASE), không dùng vòng lặp.
- Đọc biến Temp chưa được gán trong chu kỳ này.

Tự thực hành

1. Trong TIA Portal tạo FC ngôn ngữ SCL tên FC_Scale theo bài mẫu, gọi trong OB1 và thử trong PLCSIM với Raw = 5530, 16589, 27648. Kết quả phải là 0,0 / 5,0 / 10,0.
2. Viết lại bài "Bơm nước bồn tự động 2 phao" bằng một lệnh gán duy nhất trong SCL, đối chiếu với lời giải ladder ở mục Mô phỏng.
3. Viết FB trình tự 3 bước bằng CASE cho bài "Trộn hóa chất theo trình tự", dùng 3 timer multi-instance.
4. Thêm giám sát: nếu ở một bước quá 10 s thì đặt #Fault và về bước 0.

Kiểm tra nhanh

1. **Trong SCL, phép gán giá trị viết thế nào?**

   A. #A = 5;
   B. #A := 5;
   C. #A == 5;
   D. MOVE 5 #A

2. **Biến chỉ được gán bên trong IF, không có ELSE. Khi điều kiện sai thì biến đó thế nào?**

   A. Về 0
   B. Giữ giá trị cũ
   C. Báo lỗi compile
   D. Đảo trạng thái

3. **Việc nào hợp với SCL hơn ladder?**

   A. Mạch liên động hai contactor
   B. Tính trung bình 10 giá trị trong một mảng
   C. Mạch tự giữ một động cơ
   D. Đèn báo theo một tiếp điểm

Đáp án: 1B, 2B, 3B

Tín hiệu analog và PID

Cảm biến analog thường xuất **4–20 mA** (chống nhiễu tốt, phát hiện được đứt dây vì 0 mA là bất thường) hoặc **0–10 V**. S7-1200 đổi giá trị đó thành số nguyên, dải chuẩn từ **0 đến 27648**.

Chuyển sang đơn vị kỹ thuật

Dùng cặp lệnh `NORM_X` (đưa về 0.0–1.0) rồi `SCALE_X` (đưa ra đơn vị thật). Ví dụ cảm biến áp suất 0–10 bar trên `%IW64`:

```
NORM_X  MIN=0  VALUE=%IW64  MAX=27648  → OUT=#Norm
SCALE_X MIN=0.0 VALUE=#Norm MAX=10.0  → OUT="Pressure_bar"
```

Khi cảm biến 4–20 mA đọc qua kênh 0–20 mA, 4 mA tương ứng khoảng 5530, nên MIN của NORM_X đặt 5530.

PID

`PID_Compact` giữ một đại lượng (nhiệt độ, áp suất, mức) bằng giá trị đặt. Gọi nó trong OB Cyclic interrupt, cấu hình đầu vào/ra, rồi dùng chức năng tự chỉnh (tuning) trong TIA Portal.

> **Đồ án:** điều khiển mức nước hoặc nhiệt độ bằng PID là đề tài kinh điển, có giá trị khi phỏng vấn vì thể hiện bạn hiểu cả analog lẫn điều khiển vòng kín.

Các điểm mốc của tín hiệu

Tín hiệu	Giá trị số	Ý nghĩa
0 mA hoặc 0 V	0	Đầu dải kiểu 0–20 mA / 0–10 V
4 mA	5530	Đầu dải kiểu 4–20 mA (trên kênh 0–20 mA)
20 mA hoặc 10 V	27648	100% dải đo
Trên 27648 đến 32511		Vùng vượt dải, vẫn đọc được
32767		Tràn hoặc đứt dây (tùy module)

Hai kênh analog tích hợp của 1214C là **0–10 V**. Muốn đọc 4–20 mA cần module SM 1231 hoặc signal board, hoặc mắc điện trở 500 Ω để đổi dòng thành áp (20 mA × 500 Ω = 10 V).

Bài tính mẫu

Cảm biến mức 0–5 m, 4–20 mA, đọc được 16589. Chuẩn hóa: (16589 − 5530) / (27648 − 5530) = 0,5. Mức = 0,5 × 5 m = 2,5 m. Kiểm tra ngược: 2,5 m ứng với 12 mA, đúng giữa dải.

Ngõ ra analog và cảnh báo

Xuất tốc độ cho biến tần: NORM_X (0–50 Hz) rồi SCALE_X ra 0–27648 và ghi vào `%QW`. Nên có 4 ngưỡng cảnh báo HH, H, L, LL với độ trễ (hysteresis) vài phần trăm để cảnh báo không nhấp nháy quanh ngưỡng. Tín hiệu rung thì lọc bằng trung bình trượt hoặc bộ lọc bậc nhất.

PID trong vài dòng

P phản ứng theo sai lệch hiện tại, I khử sai lệch tĩnh, D phản ứng theo tốc độ thay đổi (thường để nhỏ hoặc tắt với mức và áp suất). PID_Compact phải được gọi trong OB ngắt chu kỳ (ví dụ 100 ms) để thời gian lấy mẫu đều.

Ví dụ thực tế

Đồng hồ áp suất trên HMI luôn hiện −2,5 bar khi máy nghỉ. Nguyên nhân: cảm biến 4–20 mA nhưng lệnh NORM_X để MIN = 0 thay vì 5530. Sửa một con số là xong, nhưng phải hiểu bảng mốc ở trên mới nhìn ra.

Lỗi người mới hay mắc

- Để MIN = 0 cho cảm biến 4–20 mA.
- Đưa Int thẳng vào SCALE_X mà không qua NORM_X.
- Đấu cảm biến dòng vào kênh áp.
- Gọi PID trong OB1 thay vì OB ngắt chu kỳ.
- Đi dây analog chung máng với cáp động lực biến tần.

Tự thực hành

1. Tính tay: cảm biến nhiệt 0–200 °C, 4–20 mA, giá trị đọc 11059. Nhiệt độ là bao nhiêu? (đáp án: 50 °C)
2. Trong TIA Portal viết FC_Scale nhận Raw, Min, Max và trả về Real, thử bằng Watch table với giá trị ở trên.
3. Viết thêm cảnh báo mức cao có độ trễ: bật khi trên 80, tắt khi dưới 75.

Kiểm tra nhanh

1. **Dải giá trị số chuẩn của ngõ vào analog S7-1200 là?**

   A. 0–1023
   B. 0–4095
   C. 0–27648
   D. 0–65535

2. **Ưu điểm của 4–20 mA so với 0–20 mA?**

   A. Rẻ hơn
   B. Phát hiện được đứt dây
   C. Đo được số âm
   D. Không cần nguồn

Đáp án: 1C, 2B

Biến tần và truyền thông công nghiệp

Biến tần thay đổi tần số để điều chỉnh tốc độ động cơ AC. Ba cách điều khiển từ PLC, từ dễ đến khó:

1. **I/O số + analog:** ngõ ra số cho chạy/dừng/đảo chiều, ngõ ra analog 0–10 V cho tốc độ. Dễ đấu, dễ dò lỗi.
2. **Modbus RTU (RS485):** ít dây, đọc được dòng, tần số, mã lỗi. S7-1200 cần module CM 1241 RS485.
3. **PROFINET:** dùng telegram tiêu chuẩn, ví dụ telegram 1 (STW1 + tốc độ đặt, ZSW1 + tốc độ thực). Cách phổ biến trong máy mới.

Trước khi chạy cần nhập thông số động cơ trên nhãn (công suất, điện áp, dòng, tần số, tốc độ) và chạy nhận dạng động cơ nếu biến tần hỗ trợ.

Truyền thông nên biết

- **PROFINET** (Siemens), **EtherNet/IP** (Rockwell), **Modbus TCP/RTU** (phổ thông).
- **S7 communication (PUT/GET)** giữa hai PLC Siemens, **OPC UA** để nối với hệ thống IT/MES.
- Kiến thức mạng cơ bản: IP, subnet mask, cùng dải mạng mới thấy nhau.

Đấu dây cơ bản một biến tần

Nguồn vào L1-L2-L3 (hoặc L-N với loại 1 pha), ra động cơ U-V-W bằng cáp có lưới chống nhiễu nối đất hai đầu. Ngõ số DI1 nhận lệnh chạy, DI2 đảo chiều, AI1 nhận 0–10 V tốc độ, một relay báo lỗi về PLC. Luôn đưa tín hiệu "biến tần sẵn sàng / lỗi" về PLC, đừng chỉ ra lệnh một chiều.

Thông số phải cài trước khi chạy

Dữ liệu động cơ theo nhãn (điện áp, dòng, công suất, tần số, tốc độ), tần số nhỏ nhất và lớn nhất, thời gian tăng tốc và giảm tốc, nguồn lệnh (bàn phím, terminal hay truyền thông), nguồn đặt tốc độ. Tăng tốc quá ngắn gây lỗi quá dòng, giảm tốc quá ngắn với tải quán tính lớn gây lỗi quá áp DC.

Telegram 1 qua PROFINET

Chiều	Word 1	Word 2
PLC → biến tần	STW1 (từ điều khiển)	NSOLL_A (tốc độ đặt, 16#4000 = 100%)
Biến tần → PLC	ZSW1 (từ trạng thái)	NIST_A (tốc độ thực)

Trình tự thường dùng: gửi `16#047E` (sẵn sàng, chưa chạy) rồi `16#047F` (chạy). Xác nhận lỗi bằng bit 7 của STW1. Bit 3 của ZSW1 báo có lỗi.

Modbus RTU trong S7-1200

Dùng `MB_COMM_LOAD` cấu hình cổng một lần, rồi `MB_MASTER` đọc/ghi thanh ghi. Mỗi thời điểm chỉ một lệnh MB_MASTER hoạt động, nên phải gọi luân phiên. Tham số tốc độ baud, parity, địa chỉ trạm phải giống nhau ở hai phía.

Ví dụ thực tế

Bơm tăng áp: cảm biến áp suất 4–20 mA về PLC, PID_Compact tính ra tốc độ, gửi cho biến tần qua analog hoặc PROFINET. Giờ thấp điểm bơm chạy 25 Hz thay vì đóng cắt liên tục, tiết kiệm điện và êm đường ống. Đây là ứng dụng biến tần phổ biến nhất bạn sẽ gặp.

Lỗi người mới hay mắc

- Đặt contactor giữa biến tần và động cơ rồi đóng cắt khi đang chạy.
- Không nhập thông số động cơ, để mặc định.
- Mở nắp biến tần ngay sau khi cắt điện khi tụ còn tích điện.
- Hai lệnh MB_MASTER chạy cùng lúc gây lỗi truyền thông.
- Quên nối đất lưới chống nhiễu của cáp động cơ.

Tự thực hành

1. Tải manual của một biến tần phổ thông (ví dụ SINAMICS V20) và tìm: thông số chọn nguồn lệnh, thời gian tăng tốc, bảng mã lỗi.
2. Tính giá trị NSOLL_A cần gửi để chạy 30 Hz khi tần số tham chiếu là 50 Hz (đáp án: 16#2666, tức 9830).
3. Vẽ sơ đồ đấu PLC với biến tần theo cách I/O số + analog.

Kiểm tra nhanh

1. **S7-1200 cần thêm gì để nói chuyện Modbus RTU với biến tần?**

   A. Không cần gì
   B. Module CM 1241 RS485
   C. Module analog
   D. Thẻ nhớ

2. **PLC có IP 192.168.0.1/24. Biến tần nào kết nối được?**

   A. 192.168.1.5
   B. 192.168.0.20
   C. 10.0.0.20
   D. 172.16.0.1

Đáp án: 1B, 2B

HMI / SCADA với WinCC

HMI là màn hình cảm ứng cạnh máy, SCADA là hệ giám sát trên máy tính cho cả nhà máy. Trong TIA Portal dùng **WinCC Basic/Comfort** cho màn KTP/TP, **WinCC Advanced/Professional** hoặc **WinCC Unified** cho PC.

Một HMI tối thiểu cần có

- **Màn tổng quan**: sơ đồ máy, trạng thái động cơ, van (màu xanh chạy, xám dừng, đỏ lỗi).
- **Màn vận hành**: Auto/Manual, Start/Stop, nhập giá trị đặt có giới hạn min/max.
- **Cảnh báo (alarm)**: mỗi lỗi một bit, có thời gian xuất hiện, xác nhận (acknowledge).
- **Xu hướng (trend)** cho analog, **phân quyền người dùng** cho thông số quan trọng.

Nguyên tắc: nút trên HMI không thay thế nút dừng khẩn cấp vật lý. Nút HMI nên ghi vào bit nhớ, PLC quyết định có chạy hay không, không ghi thẳng vào ngõ ra Q.

Kết nối và tag

Kéo đường nối giữa PLC và HMI trong *Devices & networks* để tạo HMI connection. Tag HMI trỏ tới tag PLC hoặc biến trong DB. Chu kỳ lấy mẫu mặc định 1 s, chỉ giảm xuống 100 ms cho giá trị cần mượt để không quá tải truyền thông.

Các đối tượng hay dùng

Đối tượng	Cách dùng đúng
Button	Sự kiện Press: SetBit, Release: ResetBit (nút nhấn nhả). Tránh SetBit mà không ResetBit.
I/O field	Nhập giá trị đặt, khai báo giới hạn min/max trong tag.
Hình có animation	Đổi màu theo trạng thái: xám dừng, xanh chạy, đỏ lỗi.
Alarm view	Hiện cảnh báo từ Discrete alarms (mỗi bit trong một Word là một lỗi) hoặc Analog alarms.
Trend view	Đồ thị giá trị analog theo thời gian.
Recipe	Lưu bộ thông số theo từng loại sản phẩm.

Lớp cảnh báo

Errors: máy dừng, bắt buộc xác nhận. **Warnings**: cảnh báo, máy vẫn chạy. Câu cảnh báo tốt nói rõ thiết bị và việc cần làm: "M03 Băng tải ra: quá tải, kiểm tra kẹt hàng" thay vì "Lỗi 17".

Ví dụ thực tế

Người vận hành ca đêm nhập nhầm nhiệt độ 2500 thay vì 250. Nếu I/O field có giới hạn 0–300 thì HMI từ chối ngay. Nếu thông số đó nằm sau mức đăng nhập "Kỹ thuật" thì họ còn không sửa được. Hai dòng cấu hình này tránh được một mẻ hàng hỏng.

Lỗi người mới hay mắc

- Nút HMI ghi thẳng vào ngõ ra Q, bỏ qua liên động trong PLC.
- Dùng SetBit cho nút Start mà không ResetBit: mất kết nối giữa chừng là bit kẹt ở 1.
- Màn hình quá nhiều màu và nhấp nháy, người vận hành không phân biệt được đâu là lỗi thật.
- Không giới hạn giá trị nhập và không phân quyền.

Tự thực hành

1. Trong TIA Portal thêm HMI KTP700 Basic, tạo kết nối với PLC 1214C.
2. Làm một màn hình có nút Start/Stop (nhấn nhả), đèn trạng thái, một I/O field nhập thời gian có giới hạn.
3. Tạo 3 Discrete alarms từ một Word và một Alarm view, chạy mô phỏng PLCSIM cùng HMI simulation.

Kiểm tra nhanh

1. **Nút Start trên HMI nên ghi vào đâu?**

   A. Trực tiếp vào %Q
   B. Vào bit nhớ (M hoặc DB) để PLC xử lý
   C. Vào %I
   D. Không cần ghi

2. **Chức năng nào giúp kỹ thuật biết lỗi xảy ra lúc mấy giờ?**

   A. Trend
   B. Alarm log
   C. Recipe
   D. User

Đáp án: 1B, 2B

Đi làm: quy trình dự án, dò lỗi, phỏng vấn

Một dự án tự động hóa đi qua các bước

1. Khảo sát, viết **đặc tả chức năng (FDS)**: máy làm gì, chế độ Auto/Manual, lỗi nào.
2. Lập **danh sách I/O**, chọn PLC, tính nguồn 24 V, chọn thiết bị.
3. Vẽ **bản vẽ điện**, lắp tủ, đấu dây theo bảng terminal.
4. Lập trình PLC + HMI, chạy thử bằng mô phỏng.
5. **Kiểm tra I/O (I/O check)**: từng cảm biến, từng ngõ ra, đánh dấu vào danh sách.
6. **FAT** tại xưởng, **SAT/commissioning** tại nhà máy khách, bàn giao tài liệu và bản sao lưu.

Dò lỗi theo thứ tự

Máy không chạy? Đi từ ngoài vào trong: **nguồn** (đèn nguồn 24 V, aptomat) → **đèn LED ngõ vào** trên PLC → **online xem ladder** network nào đang chặn → **LED ngõ ra** → relay/contactor → tải. Đa số lỗi nằm ở cảm biến lệch, dây lỏng, cầu chì, không phải ở chương trình.

Câu hỏi phỏng vấn thực tập hay gặp

- Chu kỳ quét PLC là gì? Double coil là gì?
- PNP và NPN khác nhau thế nào, đấu vào S7-1200 ra sao?
- Vì sao nút Stop đấu NC?
- TON, TOF, TP khác nhau thế nào? Cho ví dụ ứng dụng.
- FC và FB khác nhau ở đâu?
- 4–20 mA đổi sang đơn vị kỹ thuật thế nào?
- Kể về đồ án/dự án bạn đã làm: bạn làm phần nào, gặp lỗi gì, sửa ra sao.

Hồ sơ nên có: 2–3 project TIA Portal tự làm (có tag table, comment, HMI), video ngắn chạy mô phỏng, bản vẽ điện tự vẽ. Đây là bằng chứng mạnh hơn bảng điểm.

Tài liệu bạn sẽ được yêu cầu làm

Tài liệu	Nội dung
I/O list	Địa chỉ, tên tag, mô tả, loại tín hiệu, vị trí terminal, trang bản vẽ.
Biên bản I/O check	Từng điểm I/O: đã thử, ai thử, ngày, ghi chú.
Danh sách cảnh báo	Mã, nội dung, nguyên nhân, cách xử lý.
Nhật ký sửa đổi	Ngày, người sửa, sửa gì, phiên bản sao lưu.
Hướng dẫn vận hành	Khởi động, dừng, xử lý lỗi thường gặp cho người đứng máy.

Bảng dò lỗi nhanh

Hiện tượng	Kiểm tra theo thứ tự
Nhấn Start máy không chạy	LED ngõ vào Start có sáng → online xem network nào hở → E-Stop, cửa an toàn → chế độ Auto/Manual → lỗi chưa reset
LED ngõ ra sáng nhưng tải không chạy	Điện áp tại terminal ngõ ra → relay trung gian → cầu chì → cuộn contactor → relay nhiệt
Cảm biến không báo	LED trên cảm biến → khoảng cách, bụi → nguồn 24 V tại đầu dây → dây tín hiệu → kênh ngõ vào (thử đổi kênh)
CPU về STOP	Diagnostics buffer → lỗi module → vượt thời gian chu kỳ → truy cập vùng nhớ không tồn tại
Lỗi chập chờn	Đầu cốt lỏng → cáp gãy ngầm ở xích dẫn → nhiễu từ biến tần → nguồn 24 V sụt áp

Cách kể về đồ án khi phỏng vấn

Theo trình tự: bài toán là gì, bạn chọn thiết bị nào và vì sao, cấu trúc chương trình (khối nào làm gì), một lỗi bạn gặp và cách tìm ra, nếu làm lại bạn sẽ cải thiện gì. Nhà tuyển dụng không chờ đề tài lớn, họ muốn thấy bạn hiểu thứ mình đã làm.

Ví dụ thực tế

Ngày đầu thực tập, bạn thường được giao: dán nhãn dây, kiểm tra I/O theo danh sách, cập nhật bản vẽ sau sửa đổi, sao lưu chương trình. Làm cẩn thận những việc này là cách nhanh nhất để được giao việc lập trình.

Lỗi người mới hay mắc

- Sửa chương trình trên máy khách mà không ghi nhật ký và không sao lưu.
- Đoán lỗi ở chương trình trong khi chưa nhìn LED ngõ vào.
- Force một tín hiệu an toàn để "chạy tạm".
- Trả lời phỏng vấn bằng định nghĩa thuộc lòng thay vì ví dụ mình đã làm.

Tự thực hành

1. Chọn một bài tập trong app và viết đủ bộ tài liệu cho nó: I/O list, danh sách 3 cảnh báo, hướng dẫn vận hành nửa trang.
2. Tự trả lời thành tiếng 7 câu hỏi phỏng vấn ở trên, mỗi câu dưới 1 phút, có ví dụ.
3. Quay video 2 phút chạy thử một bài trong PLCSIM và giải thích từng network.

Kiểm tra nhanh

1. **Máy không chạy, bước dò lỗi nào nên làm đầu tiên?**

   A. Download lại chương trình
   B. Kiểm tra nguồn và LED ngõ vào
   C. Thay CPU
   D. Viết lại chương trình

2. **FAT là gì?**

   A. Kiểm tra nghiệm thu tại xưởng nhà sản xuất
   B. Một loại cảm biến
   C. Lỗi phần cứng
   D. Chuẩn truyền thông

Đáp án: 1B, 2A